PENTESTGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing

Gelei Deng[1], Yi Liu[1], Víctor Mayoral-Vilches[23], Peng Liu[4], Yuekang Li[5], Yuan Xu[1], Tianwei Zhang[1], Yang Liu[1], Martin Pinzger[3], Stefan Rass[6]

[1]Nanyang Technological University, [2]Alias Robotics, [3]Alpen-Adria-Universität Klagenfurt, [4]Institute for Infocomm Research (I^2R), A*STAR, Singapore, [5]University of New South Wales, [6]Johannes Kepler University Linz

Abstract

Penetration testing, a crucial industrial practice for ensuring system security, has traditionally resisted automation due to the extensive expertise required by human professionals. Large Language Models (LLMs) have shown significant advancements in various domains, and their emergent abilities suggest their potential to revolutionize industries. In this work, we establish a comprehensive benchmark using real-world penetration testing targets and further use it to explore the capabilities of LLMs in this domain. Our findings reveal that while LLMs demonstrate proficiency in specific sub-tasks within the penetration testing process, such as using testing tools, interpreting outputs, and proposing subsequent actions, they also encounter difficulties maintaining a whole context of the overall testing scenario.

Based on these insights, we introduce PENTESTGPT, an LLM-empowered automated penetration testing framework that leverages the abundant domain knowledge inherent in LLMs. PENTESTGPT is meticulously designed with three self-interacting modules, each addressing individual sub-tasks of penetration testing, to mitigate the challenges related to context loss. Our evaluation shows that PENTESTGPT not only outperforms LLMs with a task-completion increase of 228.6% compared to the GPT-3.5 model among the benchmark targets, but also proves effective in tackling real-world penetration testing targets and CTF challenges. Having been open-sourced on GitHub, PENTESTGPT has garnered over 6,200 stars in 9 months and fostered active community engagement, attesting to its value and impact in both the academic and industrial spheres.

1 Introduction

Securing a system presents a formidable challenge. Offensive security methods like penetration testing (pen-testing) and red teaming are now essential in the security lifecycle. As explained by Applebaum [1], these approaches involve security teams attempting breaches to reveal vulnerabilities, providing advantages over traditional defenses, which rely on incomplete system knowledge and modeling. This study, guided by the principle *"the best defense is a good offense"*, focuses on offensive strategies, specifically penetration testing.

Penetration testing is a proactive offensive technique for identifying, assessing, and mitigating security vulnerabilities [2]. It involves targeted attacks to confirm flaws, yielding a comprehensive inventory of vulnerabilities with actionable recommendations. This widely-used practice empowers organizations to detect and neutralize network and system vulnerabilities before malicious exploitation. However, it typically relies on manual effort and specialized knowledge [3], resulting in a labor-intensive process, creating a gap in meeting the growing demand for efficient security evaluations.

Large Language Models (LLMs) have demonstrated profound capabilities, showcasing intricate comprehension of human-like text and achieving remarkable results across a multitude of tasks [4, 5]. An outstanding characteristic of LLMs is their emergent abilities [6], cultivated during training, which empower them to undertake intricate tasks such as reasoning, summarization, and domain-specific problem-solving without task-specific fine-tuning. This versatility posits LLMs as potential game-changers in various fields, notably cybersecurity. Although recent works [7–9] posit the potential of LLMs to reshape cybersecurity practices, including the context of penetration testing, there is an absence of a systematic, quantitative assessment of their aptitude in this regard. Consequently, an imperative question presents: To what extend can LLMs automate penetration testing?

Motivated by this question, we set out to explore the capability boundary of LLMs on real-world penetration testing tasks. Unfortunately, the current benchmarks for penetration testing [10, 11] are not comprehensive and fail to assess progressive accomplishments fairly during the process. To address this limitation, we construct a robust benchmark that includes test machines from HackTheBox [12] and VulnHub [13]—two leading platforms for penetration testing challenges. Comprising 13 targets with 182 sub-tasks, our benchmark encompasses all vulnerabilities appearing in

OWASP's top 10 vulnerability list [14] and 18 Common Weakness Enumeration (CWE) items [15]. The benchmark offers a more detailed evaluation of the tester's performance by monitoring the completion status for each sub-task.

With this benchmark, we perform an exploratory study using GPT-3.5 [16], GPT-4 [17], and Bard [18] as representative LLMs. Our test strategy is interactive and iterative. We craft tailored prompts to guide the LLMs through penetration testing. Each LLM, presented with prompts and target machine information, generates step-by-step penetration testing operations. We then execute the suggested operations in a controlled environment, document the results, and feed them back to the LLM to inform and refine its next steps. This cycle (prompting, executing, and feedback) is repeated until the LLM completes the entire penetration testing process autonomously. To evaluate LLMs, we compare their results against baseline solutions from official walkthroughs and certified penetration testers. By analyzing similarities and differences in their problem-solving approaches, we aim to better understand LLMs' capabilities in penetration testing and how their strategies differ from human experts.

Our investigation yields intriguing insights into the capabilities and limitations of LLMs in penetration testing. We discover that LLMs demonstrate proficiency in managing specific sub-tasks within the testing process, such as utilizing testing tools, interpreting their outputs, and suggesting subsequent actions. Compared to human experts, LLMs are especially adept at executing complex commands and options with testing tools, while models like GPT-4 excel in comprehending source code and pinpointing vulnerabilities. Furthermore, LLMs can craft appropriate test commands and accurately describe graphical user-interface operations needed for specific tasks. Leveraging their vast knowledge base, they can design inventive testing procedures to unveil potential vulnerabilities in real-world systems and CTF challenges. However, we also note that LLMs have difficulty in maintaining a coherent grasp of the overarching testing scenario, a vital aspect for attaining the testing goal. As the dialogue advances, they may lose sight of earlier discoveries and struggle to apply their reasoning consistently toward the final objective. Additionally, LLMs overemphasize recent tasks in the conversation history, regardless of their vulnerability status. As a result, they tend to neglect other potential attack surfaces exposed in prior tests and fail to complete the penetration testing task.

Building on our insights into LLMs' capabilities in penetration testing, we present PENTESTGPT[1], an interactive system designed to enhance the application of LLMs in this domain. Drawing inspiration from the collaborative dynamics commonly observed in real-world human penetration testing teams, PENTESTGPT is particularly tailored to manage large and intricate projects. It features a tripartite architecture comprising *Reasoning, Generation, and Parsing Modules*, each reflecting specific roles within penetration testing teams. The Reasoning Module emulates the function of a lead tester, focusing on maintaining a high-level overview of the penetration testing status. We introduce a novel representation, the Pentesting Task Tree (PTT), based on the cybersecurity attack tree [19]. This structure encodes the testing process's ongoing status and steers subsequent actions. Uniquely, this representation can be translated into natural language and interpreted by the LLM, thereby comprehended by the Generation Module and directing the testing procedure. The Generation Module, mirroring a junior tester's role, is responsible for constructing detailed procedures for specific sub-tasks. Translating these into exact testing operations augments the generation process's accuracy. Meanwhile, the Parsing Module deals with diverse text data encountered during penetration testing, such as tool outputs, source codes, and HTTP web pages. It condenses and emphasizes these texts, extracting essential information. Collectively, these modules function as an integrated system. PENTESTGPT completes complex penetration testing tasks by bridging high-level strategies with precise execution and intelligent data interpretation, thereby maintaining a coherent and effective testing process.

We assessed PENTESTGPT across diverse testing scenarios to validate its effectiveness and breadth. In our custom benchmarks, PENTESTGPT significantly outperformed direct applications of GPT-3.5 and GPT-4, showing increases in sub-task completion rates of 228.6% and 58.6%, respectively. Furthermore, when applied to real-world challenges such as the HackTheBox active machine penetration tests [20] and picoMini [21] CTF competition, PENTESTGPT demonstrated its practical utility. It successfully resolved 4 out of 10 penetration testing challenges, incurring a total cost of 131.5 US Dollars for the OpenAI API usage. In the CTF competition, PENTESTGPT achieved a score of 1500 out of a possible 4200, placing 24th among 248 participating teams. This evaluation underscores PENTESTGPT's practical value in enhancing penetration testing tasks' efficiency and precision. The solution has been made publicly available on GitHub[2], receiving widespread acclaim with over 6,200 stars to the date of writing, active community engagement, and ongoing collaboration with multiple industrial partners.

As a long term research goal, we aim to contribute to unlocking the potential of modern machine learning approaches and develop a fully automated penetration testing framework that helps produce cybersecurity cognitive engines. Our overall architecture is depicted in Figure 1, showing our current work and future planned contributions. Our proposed framework, MALISM, is designed to enable a user without in-depth security domain knowledge to produce its cybersecurity cognitive engine that helps conduct penetration testing over an extensive range of targets. This framework comprises three primary components:

[1] PENTESTGPT is King Arthur's legendary sword, known for its exceptional cutting power and the ability to pierce armor.

[2] The project is at: https://github.com/GreyDGL/PentestGPT.

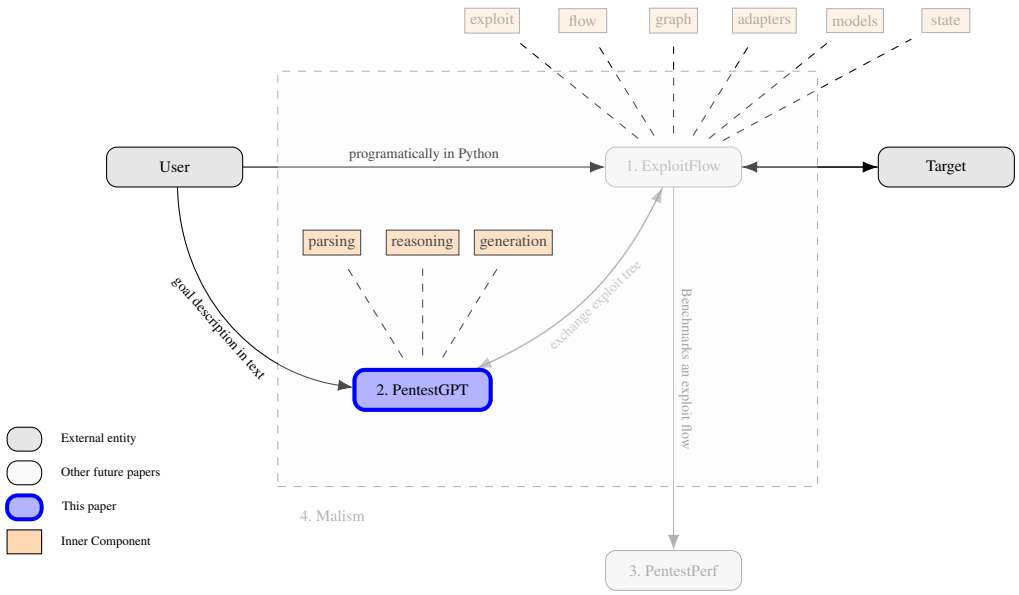

Figure 1: Architecture of our framework to develop a fully automated penetration testing tools, MALISM. Figure depicts the various interaction flows that an arbitrary `User` could follow using MALISM to pentest a given `Target`. **1.** Corresponds with EXPLOITFLOW, a modular library to produce security exploitation routes (*exploit flows*) that caputures the state of the system being tested in a flow after every discrete action. **2. (this paper)** Corresponds with PENTESTGPT, a testing tool that leverages the power of LLMs to produce testing guidance (heuristics) for every given discrete state. **3.** PENTESTPERF is a comprehensive penetration testing benchmark to evaluate the performances of penetration testers and automated tools across a wide array of testing targets. **4.** captures MALISM, our framework to develop fully automated penetration testing tools which we name *cybersecurity cognitive engines*.

1. EXPLOITFLOW [22]: A modular library to produce cyber security exploitation routes (*exploit flows*). EXPLOITFLOW aims to combine and compose exploits from different sources and frameworks, capturing the state of the system being tested in a flow after every discrete action, which allows learning attack trees that affect a given system. EXPLOITFLOW's main motivation is to facilitate and empower Game Theory and Artificial Intelligence (AI) research in cyber security. It uniquely represents the exploitation process that encodes every facet within it. Its representation can be effectively integrated with various penetration testing tools and scripts, such as Metasploit [23] to perform end-to-end penetration testing. Such representation can be further visualized to guide the human experts to reproduce the testing process.

2. PENTESTGPT (**this paper**): An automated penetration testing system that leverages the power of LLMs to produce testing guidance and intuition at every given discrete state. It functions as the core component of the MALISM framework, guiding the LLMs to utilize their domain knowledge in real-world testing scenarios efficiently.

3. PENTESTPERF: A comprehensive penetration testing benchmark developed to evaluate the performances of penetration testers and automated tools across a wide array of testing targets. It offers a fair and robust platform for performance comparison.

The harmonious integration of these three components forms an automated, self-evolving penetration testing framework capable of executing penetration tests over various targets, MALISM. This framework to develop fully automated penetration testing tools, which we name *cybersecurity cognitive engines*, aims to revolutionize the field of penetration testing by significantly reducing the need for domain expertise and enabling more comprehensive and reliable testing.

In summary, we make the following contributions:

- **Development of a Comprehensive Penetration Testing Benchmark.** We craft a robust and representative penetration testing benchmark, encompassing a multitude of test machines from leading platforms such as HackTheBox and VulnHub. This benchmark includes 182 sub-tasks covering OWASP's top 10 vulnerabilities, offering fair and comprehensive evaluation of penetration testing. To the best of

our knowledge, this is the first benchmark in the field that can provide progressive accomplishments assessments and comparisons.

- **Comprehensive Evaluation of LLMs for Penetration Testing Tasks.** By employing models like GPT-3.5, GPT-4, and Bard, our exploratory study rigorously investigates the strengths and limitations of LLMs in penetration testing. To the best of our knowledge, this is the first systematic and quantitative study for the capability of LLMs in performing automated penetration testing. The insights gleaned from this study shed valuable light on the capabilities and challenges faced by LLMs, enriching our understanding of their applicability in this specialized domain.

- **Development of an Innovative LLM-powered Penetration Testing System.** We engineer PENTESTGPT, a novel interactive system that leverages the strengths of LLMs to carry out penetration testing tasks automatically. Drawing inspiration from real-world human penetration testing teams, PENTESTGPT integrates a tripartite design that mirrors the collaborative dynamics between senior and junior testers. This architecture optimizes LLMs' usage, significantly enhancing the efficiency and effectiveness of automated penetration testing. We have open-sourced PENTESTGPT and it has received over 6,500 stars on GitHub, active community contributions, and industry partners including AWS, Huawei, and ByteDance to collaborate.

2 Background & Related Work

2.1 Penetration Testing

Penetration testing, or "pentesting", is a critical practice to enhance organizational systems' security. In a typical penetration test, security professionals, known as penetration testers, analyze the target system, often leveraging automated tools. The standard process is divided into five key phases [24]: Reconnaissance, Scanning, Vulnerability Assessment, Exploitation, and Post Exploitation (including reporting). These phases enable testers to understand the target system, identify vulnerabilities, and exploit them to gain access.

Despite significant advancements [11, 25, 26], a fully automated penetration testing system remains out of reach. This gap results from the need for deep vulnerability understanding and a strategic action plan. Typically, testers combine depth-first and breadth-first search techniques [24]. They first grasp the target environment's scope, then drill down into specific vulnerabilities. This method ensures comprehensive analysis, leaning on expertise and experience. The multitude of specialized tools further complicate the automation. Thus, even with artificial intelligence, achieving a seamless automated penetration testing solution is a daunting task.

2.2 Large Language Models

Large Language Models (LLMs), including OpenAI's GPT-3.5 and GPT-4, are prominent tools with applications extending to various cybersecurity-related fields, such as code analysis [27] and vulnerability repairment [28]. These models are equipped with wide-ranging general knowledge and the capacity for elementary reasoning. They can comprehend, infer, and produce text resembling human communication, aided by a training corpus encompassing diverse domains like computer science and cybersecurity. Their ability to interpret context and recognize patterns enables them to adapt knowledge to new scenarios. This adaptability, coupled with their proficiency in interacting with systems in a human-like way, positions them as valuable assets in enhancing penetration testing processes. Despite inherent limitations, LLMs offer distinct attributes that can substantially aid in the automation and improvement of penetration testing tasks. The realization of this potential, however, requires the creation and application of a specialized and rigorous benchmark.

3 Penetration Testing Benchmark

3.1 Motivation

The comprehensive evaluation of LLMs in penetration testing necessitates a robust and representative benchmark. Existing benchmarks in this domain [10, 11] have several limitations. First, they are often restricted in scope, focusing on a narrow range of potential vulnerabilities, and thus fail to capture the complexity and diversity of real-world cyber threats. For instance, the OWASP *juiceshop* project [29] is the most widely adopted benchmark for web vulnerability evaluation. However, it does not include privilege escalation vulnerabilities, which is an essential aspect of penetration testing. Second, existing benchmarks may not recognize the cumulative value of progress through the different stages of penetration testing, as they tend to evaluate only the final exploitation success. This approach overlooks the nuanced value each step contributes to the overall process, resulting in metrics that might not accurately represent actual performance in real-world scenarios.

To address these concerns, we propose the construction of a comprehensive penetration testing benchmark that meets the following criteria:

Task Variety. The benchmark must encompass diverse tasks, reflecting various operating systems and emulating the diversity of scenarios encountered in real-world penetration testing.

Challenge Levels. To ensure broad applicability, the benchmark must include tasks of varying difficulty levels suitable for challenging novice and expert testers.

Progress Tracking. Beyond mere success or failure metrics, the benchmark must facilitate tracking of incremental progress, thereby recognizing and scoring the value added at

each stage of the penetration testing process.

3.2 Benchmark Design

Following the criteria outlined previously, we develop a comprehensive benchmark that closely reflects real-world penetration testing tasks. The design process progresses through several stages.

Task Selection. We begin by selecting tasks from HackTheBox [12] and VulnHub [13], two leading penetration testing training platforms. Our selection criteria are designed to ensure that our benchmark accurately reflects the challenges encountered in practical penetration testing environments. We meticulously review the latest machines available on both platforms, aiming to identify and select a subset that comprehensively covers all vulnerabilities listed in the OWASP [14] Top 10 Project. Additionally, we choose machines that represent a mix of difficulties, classified according to traditional standards in the penetration testing domain into *easy*, *medium*, and *hard* categories. This process guarantees that our benchmark spans the full spectrum of vulnerabilities and difficulties. Note that our benchmark does not include benign targets to assess false positives. In penetration testing, benign targets are sometimes explored. Our main objective remains identifying true vulnerabilities.

Task Decomposition. We further parse the testing process of each target into a series of sub-tasks, following the standard solution commonly referred to as the "walkthrough" in penetration testing. Each sub-task corresponds to a unique step in the overall process. We decompose sub-tasks following NIST 800-115 [30], the Technical Guide to Security Testing. Each sub-task is one step declared in the Guide (e.g., network discovery, password cracking), or an operation that exploits a unique vulnerability categorised in the Common Weakness Enumeration (CWE) [15] (e.g., exploiting SQL injection - CWE-89 [31]). In the end, we formulate an exhaustive list of sub-tasks for every benchmark target. We provide the complete list of the decomposed sub-tasks in Appendix Table 7.

Benchmark Validation. The final stage of our benchmark development involves rigorous validation, which ensures the reproducibility of these benchmark machines. To do this, three certified penetration testers independently attempt the penetration testing targets and write their walkthrough. We then adjust our task decomposition accordingly because some targets may have multiple valid solutions.

Ultimately, we have compiled a benchmark that effectively encompasses all types of vulnerabilities listed in the OWASP [14] Top 10 Project. It comprises 13 penetration testing targets, each at varying levels of difficulty. These targets are broken down into 182 sub-tasks across 26 categories, covering 18 distinct CWE items. This number of targets is deemed sufficient to represent a broad spectrum of vulnerabilities, difficulty levels, and varieties essential for comprehensive penetration testing training. Detailed information about the included categories can be found in the Appendix Section 7. To foster community development, we have made this benchmark publicly available online at our anonymous project website [32].

4 Exploratory Study

We conduct an exploratory study to assess the capabilities of LLMs in penetration testing, with the primary objective of determining how well LLMs can adapt to the real-world complexities and challenges in this task. Specifically, we aim to address the following two research questions:

RQ1 (Capability): To what extent can LLMs perform penetration testing tasks?

RQ2 (Comparative Analysis): How do the problem-solving strategies of human penetration testers and LLMs differ?

We utilize the benchmark described in Section 3 to evaluate the performance of LLMs on penetration testing tasks. In the following, we first delineate our testing strategy for this study. Subsequently, we present the testing results and an analytical discussion to address the above research questions.

4.1 Testing Strategy

LLMs are text-based and cannot independently perform penetration testing operations. To address this, we develop a human-in-the-loop testing strategy, serving as an intermediary method to accurately assess LLMs' capabilities. This strategy features an interactive loop where a human expert executes the LLM's penetration testing directives. Importantly, the human expert functions purely as an executor, strictly following the LLM's instructions without adding any expert insights or making independent decisions.

Figure 2 decipits the testing strategy with the following steps: ❶ We initiate the looped testing procedure by presenting the target specifics to the LLM, seeking its guidance on potential penetration testing steps. ❷ The human expert strictly follows the LLM's recommendations and conducts the suggested actions in the penetration testing environment. ❸ Outcomes of the testing actions are collected and summarized: direct text outputs such as terminal outputs or source code are documented; non-textual results, such as graphical representations, are translated by the human expert into succinct textual summaries. The data is then fed back to the LLM, setting the stage for its subsequent recommendations. ❹ This iterative process persists either until a conclusive solution is identified or an deadlock is reached. We then compile a record of the testing procedures, encompassing successful sub-tasks, ineffective actions, and any reasons for failure, if applicable. For a more tangible grasp of this strategy, we offer illustrative examples of prompts and corresponding outputs from GPT-4 related to one of our benchmark targets in the Appendix Section A.